

Теоретический материал: "Множества: создание, операции над множествами, методы"

1. Что такое множества?

Множество - это структура данных в Python, которая хранит уникальные элементы без сохранения порядка.

Множества поддерживают быстрый доступ к данным и выполнение математических операций.

Основные свойства:

- Все элементы уникальны.
- Элементы неупорядочены (доступ по индексу невозможен).

2. Создание множества

- С помощью литерала {}:

```
fruits = {"яблоко", "банан", "апельсин"}  
print(fruits) # {"яблоко", "банан", "апельсин"}
```

- С помощью функции set():

```
numbers = set([1, 2, 3, 4])  
print(numbers)
```

- Пустое множество:

```
empty_set = set() # {} создает пустой словарь
```

3. Основные операции над множествами

- Объединение (| или union()):
Возвращает множество, содержащее все элементы обоих множеств.

```
set1 = {1, 2, 3}  
set2 = {3, 4, 5}  
print(set1 | set2) # {1, 2, 3, 4, 5}
```

- Пересечение (& или intersection()):
Возвращает множество, содержащее элементы, общие для обоих множеств.

```
print(set1 & set2) # {3}
```

- Разность (- или difference()):
Возвращает множество, содержащее элементы из первого множества, которых нет во втором.

```
print(set1 - set2) # {1, 2}
```

- Симметрическая разность (^ или symmetric_difference()):
Возвращает множество, содержащее элементы, присутствующие в одном из множеств, но не в обоих.

```
print(set1 ^ set2) # {1, 2, 4, 5}
```

4. Методы множеств

- add(элемент):
Добавляет элемент в множество.

```
fruits.add("груша")  
print(fruits) # {"яблоко", "банан", "апельсин", "груша"}
```

- remove(элемент):
Удаляет элемент из множества. Если элемента нет, вызывает ошибку.

```
fruits.remove("яблоко")
```

- discard(элемент):
Удаляет элемент, если он есть. Не вызывает ошибку, если элемента нет.

```
fruits.discard("ананас")
```

- `pop()`:
Удаляет и возвращает случайный элемент множества.

```
random_fruit = fruits.pop()
print(random_fruit)
```

- `clear()`:
Очищает множество.

```
fruits.clear()
```

- `isdisjoint(другое_множество)`:
Возвращает `True`, если множества не имеют общих элементов.

```
print(set1.isdisjoint(set2)) # False
```

- `issubset(другое_множество)`:
Проверяет, является ли множество подмножеством другого.

```
print({1, 2}.issubset(set1)) # True
```

- `issuperset(другое_множество)`:
Проверяет, является ли множество надмножеством другого.

```
print(set1.issuperset({1, 2})) # True
```

5. Примеры задач

Пример 1: Операции над множествами

```
set1 = {1, 2, 3, 4}
set2 = {3, 4, 5, 6}

print("Объединение", set1 | set2) # {1, 2, 3, 4, 5, 6}
print("Пересечение", set1 & set2) # {3, 4}
print("Разность", set1 - set2) # {1, 2}
```

```
print("Симметрическая разность:", set1 ^ set2) # {1, 2, 5, 6}
```

Пример 2: Удаление дубликатов из списка

```
numbers = [1, 2, 2, 3, 4, 4, 5]
unique_numbers = set(numbers)
print(unique_numbers) # {1, 2, 3, 4, 5}
```

Пример 3: Фильтрация уникальных символов в строке

```
text = "hello world"
unique_chars = set(text)
print(unique_chars) # {'h', 'e', 'l', 'o', ' ', 'w', 'r', 'd'}
```

Задача 1: Новогодние украшения

На складе магазина хранятся украшения: шары, гирлянды и звёзды. Все они представлены в виде множества decorations.

1. Создайте это множество.
2. Добавьте к нему элемент "снежинки".
3. Удалите элемент "звёзды".

Задача 2: Группы студентов

Есть две группы студентов: одна изучает Python, другая - Java. Состав групп записан во множествах.

1. Выведите общий список студентов (объединение).
2. Найдите студентов, которые изучают оба языка (пересечение).
3. Определите студентов, изучающих только Python.

Пример данных:

```
python_group = {"Алиса", "Борис", "Катя"}
java_group = {"Катя", "Дима", "Елена"}
```

6. Итоги

- **Множества** - это удобный инструмент для работы с уникальными данными.

- Поддерживают математические операции: объединение, пересечение, разность, симметрическую разность.
- Методы множеств позволяют добавлять, удалять, фильтровать и проверять элементы.